

API Testing with Postman

A. API Basics & Fundamentals

1. What is API testing and why is it important?

API testing is the process of verifying that Application Programming Interfaces work as intended — not just in terms of returning correct data, but also in meeting functional, performance, and security requirements.

Unlike UI testing, where you check the front-end interface, API testing focuses on the business logic layer — ensuring that endpoints correctly process requests and return expected responses.

It's important because:

- APIs are the backbone of most modern applications (especially microservices).
- Issues at the API level can break multiple downstream systems.
- Faster feedback: API tests execute quicker than UI tests.

Example: In an e-commerce app, the **Add to Cart API** must correctly handle stock availability, pricing, and user authentication before the UI can display it.

2. What are the different types of APIs?

- **Public APIs** – Available to any developer (e.g., Google Maps API).
- **Private APIs** – Internal to an organization.
- **Partner APIs** – Shared with specific business partners under agreements.
- **Composite APIs** – Combine multiple APIs into one call (common in microservices).

Knowing API type is important because it influences authentication, security, and testing scope.

3. What is the difference between SOAP and REST APIs?

SOAP (Simple Object Access Protocol)

- Uses XML format for requests/responses.
- Strict contract (WSDL).
- Heavy, more secure by default.
- Suitable for complex enterprise systems.

REST (Representational State Transfer)

- Uses HTTP methods (GET, POST, PUT, DELETE).
- Can return JSON, XML, HTML.
- Lightweight and faster.
- Most popular for modern web/mobile apps.

4. Explain HTTP methods in API testing.

- **GET** – Retrieve data from the server.
- **POST** – Create new resources.
- **PUT** – Update a resource completely.
- **PATCH** – Update a resource partially.
- **DELETE** – Remove a resource.

Example:

GET /users/1 – fetch user with ID 1

PUT /users/1 – update the entire user object

PATCH /users/1 – update only part of the user data

5. What is idempotency in HTTP methods?

Idempotency means sending the same request multiple times will result in the same outcome.

- **GET, PUT, DELETE** → Idempotent.
- **POST** → Not idempotent.

Example:

- PUT /user/1 { "name": "John" } → No matter how many times you send it, result remains the same.
 - POST /user { "name": "John" } → Multiple sends will create multiple users.
-

6. Difference between URI, URL, and Endpoint.

- **URI** – Identifier for a resource (more general).
- **URL** – A type of URI that includes the location (e.g., protocol + domain + path).
- **Endpoint** – Specific URL where an API resource can be accessed.

Example:

- URI: /users/123
 - URL: https://api.example.com/users/123
 - Endpoint: GET request to that URL.
-

7. Categories of HTTP status codes.

- **1xx** – Informational (100 Continue)
- **2xx** – Success (200 OK, 201 Created)
- **3xx** – Redirection (301 Moved Permanently)
- **4xx** – Client Error (400 Bad Request, 401 Unauthorized, 404 Not Found)

- **5xx** – Server Error (500 Internal Server Error)
-

8. What are headers in API requests and responses?

Headers carry metadata — such as authentication tokens, content type, caching instructions.

Examples:

- `Content-Type: application/json`
 - `Authorization: Bearer <token>`
 - `Accept-Language: en-US`
-

9. What is JSON and why is it preferred over XML?

JSON (JavaScript Object Notation) is lightweight, easy to read, and natively supported by JavaScript.

Compared to XML:

- Smaller payloads.
 - No closing tags → faster parsing.
 - More readable for humans.
-

10. How do you validate an API response?

Validation checks:

- **Status Code** – Expected 200, 201, etc.
 - **Response Body** – Match expected fields and values.
 - **Headers** – Verify content type, caching.
 - **Schema Validation** – Ensure response matches defined structure.
-

B. Postman Features & Usage

11. What is Postman and why is it widely used in API testing?

Postman is an API development and testing tool that allows testers and developers to send requests to APIs, analyze responses, and automate test scenarios.

Why it's popular:

- **User-friendly UI** – Even beginners can quickly send API requests.
- **Supports multiple protocols** – HTTP, HTTPS, WebSocket, gRPC.
- **Built-in scripting** – Pre-request and test scripts using JavaScript.
- **Automation support** – Newman CLI allows integration with CI/CD.
- **Environment & variable support** – Useful for switching between dev/staging/prod.

Example: Instead of manually writing curl commands, you can test an API with a click in Postman and save it for re-use.

12. Explain the structure of a request in Postman.

A request in Postman typically consists of:

- **Method** (GET, POST, PUT, DELETE, PATCH)
 - **URL/Endpoint** (e.g., `https://api.example.com/users`)
 - **Headers** (e.g., `Content-Type: application/json`)
 - **Body** (Raw JSON, Form-data, GraphQL query, etc.)
 - **Authorization** (Basic, Bearer Token, OAuth, etc.)
 - **Params** (Query parameters like `?page=2&limit=10`)
-

13. What are Postman Collections and why are they important?

A **Collection** is a folder that organizes related API requests into a single group.

Importance:

- Helps keep test cases grouped logically.
- Supports sharing across teams.
- Can be run as a batch for automation.
- Supports documentation generation.

Example: You can create a **User Management** collection containing APIs for creating, updating, deleting, and retrieving users.

14. What are environments in Postman?

Environments in Postman allow you to store and switch between sets of variables for different API contexts (Dev, Staging, Production).

Example:

- **Dev Base URL:** `https://dev.api.example.com`
- **Prod Base URL:** `https://api.example.com`

By changing the environment, you can run the same requests against different servers without editing the requests manually.

15. What are variables in Postman and how do you use them?

Variables store reusable values to make tests dynamic.

Types of variables:

- **Global** – Available everywhere.
- **Collection** – Limited to a collection.

- **Environment** – Limited to an environment.
- **Local** – Limited to a single request execution.

Usage:

```
{{baseUrl}}/users/{{userId}}
```

Here, `baseUrl` and `userId` can be stored in variables.

16. What is the difference between Pre-request Script and Tests in Postman?

- **Pre-request Script:** JavaScript code that runs **before** the request is sent.
Example: Generating a timestamp or token before request execution.
 - **Tests:** JavaScript code that runs **after** the response is received.
Example: Validating that the status code is 200.
-

17. How do you write a basic test script in Postman?

Example: Checking status code and response time:

```
pm.test("Status code is 200", function () {  
    pm.response.to.have.status(200);  
});  
  
pm.test("Response time is less than 500ms", function () {  
    pm.expect(pm.response.responseTime).to.be.below(500);  
});
```

18. How do you send authentication credentials in Postman?

Postman supports multiple authentication types:

- **Basic Auth** – Username/password encoded in Base64.
- **Bearer Token** – Token passed in Authorization header.
- **OAuth 1.0 / 2.0** – Secure authentication flows.
- **API Key** – Passed in query params or headers.

Example for Bearer Token:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

19. What is Collection Runner in Postman?

The Collection Runner allows you to run all requests in a collection in sequence.

You can:

- Pass external test data (CSV/JSON).

- Repeat iterations.
- View summary reports.

Example: Running login → add product → checkout flow in one run.

20. What is Newman and why is it used?

Newman is Postman's command-line tool to run collections in automated pipelines.

Why use it:

- Integrates Postman tests into CI/CD (Jenkins, GitLab CI).
- Supports JSON/HTML reports.
- Executes tests without the Postman GUI.

Command:

```
newman run mycollection.json -e myenv.json
```

21. How do you handle dynamic data in Postman tests?

- Use **pm.variables.set()** to set a variable dynamically.
- Store values from one response to use in another request.

Example:

```
let userId = pm.response.json().id;  
pm.environment.set("userId", userId);
```

22. How do you import/export collections and environments in Postman?

- **Export** – Right-click → Export as JSON.
 - **Import** – Click Import → Choose file or link.
- This is crucial for collaboration and version control.
-

23. What is a Monitor in Postman?

A Monitor runs your collection at scheduled intervals and sends alerts if tests fail.

Example: Run API health checks every hour and email results.

24. How do you validate JSON schema in Postman?

You can use the **tv4** or **Ajv** validator inside tests.

```
const schema = {  
  "type": "object",  
  "properties": {  
    "id": { "type": "number" },
```



```
    "name": { "type": "string" }
  },
  "required": ["id", "name"]
};

pm.test("Schema is valid", function () {
  pm.response.to.have.jsonSchema(schema);
});
```

25. How do you pass query parameters in Postman?

You can:

- Add them in the **Params** tab.
- Append to URL manually (?page=1&limit=5).
- Set dynamically with variables.

C. API Testing in Depth

26. How do you approach testing a new API when no documentation is available?

When there's no documentation, you need to **reverse-engineer** or **explore** the API:

1. **Check available code or network logs** (e.g., browser DevTools, mobile proxy tools like Charles/Fiddler).
2. **Explore using Postman** – start with GET requests to known endpoints.
3. **Check response headers and patterns** – may give hints about parameters or content type.
4. **Look at error messages** – sometimes APIs reveal required fields or validation rules.
5. **Talk to developers** – confirm assumptions before deep testing.

Example: In a mobile app, capturing network traffic during login might reveal `/auth/login` with required body parameters.

27. What are the most common API testing validations?

You should validate:

- **Status Code** – Matches expected result (e.g., 200 for success).
 - **Response Body Content** – Expected fields, values, and data types.
 - **JSON/XML Schema** – Structure matches contract.
 - **Headers** – Content-Type, caching, security headers.
 - **Performance** – Response time is within SLA.
 - **Database** – Data changes match request actions.
 - **Idempotency** – Multiple identical requests give the same result (for idempotent methods).
-

28. How do you test API error handling?

Negative testing is as important as positive testing:

- **Invalid inputs** – Missing fields, incorrect data types.
- **Unauthorized access** – Invalid token, missing token.
- **Resource not found** – Requesting an ID that doesn't exist.
- **Method not allowed** – Sending POST instead of GET.
- **Rate limiting** – Sending too many requests.

Example in Postman:

```
pm.test("Returns 404 when user not found", function () {  
    pm.expect(pm.response.code).to.eql(404);  
});
```

29. What is the difference between functional and non-functional API testing?

- **Functional Testing** – Verifies that the API does what it's supposed to (correct inputs produce correct outputs).
- **Non-functional Testing** – Focuses on performance, security, usability, scalability.

Example:

- **Functional:** Does `GET /users/1` return the correct user?
 - **Non-functional:** Does `GET /users/1` respond within 300ms under 1000 concurrent requests?
-

30. How do you validate data integrity in API testing?

Steps:

1. **Trigger the API** – e.g., `POST /users` creates a new user.
 2. **Check API Response** – Ensure the returned data matches input.
 3. **Verify Database** – Run SQL query to ensure data is stored correctly.
 4. **Retrieve via GET API** – Ensure consistency between DB and API output.
-

31. How do you test API security in Postman?

Security testing in Postman includes:

- **Authentication** – Ensure endpoints require proper login.
 - **Authorization** – Ensure users can only access permitted resources.
 - **Token Expiry** – Check that expired tokens are rejected.
 - **Data Exposure** – Ensure sensitive data (passwords, credit cards) is never in plain text.
 - **CORS Handling** – Ensure API restricts requests from unknown domains.
-

32. How do you test an API for performance using Postman?

Postman alone isn't a full performance testing tool, but you can:

- Use `pm.expect(pm.response.responseTime).to.be.below(500)` to check speed.
 - Run APIs repeatedly using **Collection Runner** with multiple iterations.
 - For deeper load testing, export Postman collections to **JMeter** or **k6**.
-

33. How do you chain API requests in Postman?

Chaining means **using output from one API as input to another**.

Example:

```
let token = pm.response.json().token;
pm.environment.set("authToken", token);
```

Then in the next request, set:

```
Authorization: Bearer {{authToken}}
```

This is useful for login flows, order placements, etc.

34. How do you test APIs that have dependencies on other services?

- Use **mock servers** when dependent services are unavailable.
- Test with **stubs** for partial functionality.
- Use **contract testing** to validate API interactions without full integration.

Example: If your payment API depends on a banking API, you can mock the bank's success/failure responses to test your service behavior.

35. How do you handle dynamic tokens or session IDs in Postman?

- Capture the token/session ID from login response:

```
let sessionId = pm.response.json().sessionId;
pm.environment.set("sessionId", sessionId);
```

- Use it in headers or params for subsequent requests:

```
Session-Id: {{sessionId}}
```

36. How do you ensure API backward compatibility?

Backward compatibility means old clients still work when API is updated.

Testing steps:

- Run old test cases on the new API version.
 - Verify deprecated fields still work (until officially removed).
 - Ensure default values are returned when new fields are added.
-

37. How do you test API pagination?

Pagination is tested by:

- **Checking correct page size** – e.g., if `limit=10`, ensure only 10 items returned.
 - **Validating total pages** – Matches total data count.
 - **Testing boundary values** – First page, last page, empty page.
-

38. How do you test sorting and filtering in APIs?

- Send API requests with sorting params (`?sort=name&order=asc`).
 - Verify the list is correctly sorted.
 - Apply filters (`?status=active`) and check only matching records are returned.
 - Combine filters and sorting for complex validations.
-

39. How do you validate API contracts?

An API contract defines **expected request/response structure** (fields, types, rules).

You can:

- Use **JSON Schema validation** in Postman.
 - Compare response with OpenAPI/Swagger definitions.
-

40. How do you test APIs that return asynchronous responses?

Some APIs return an immediate “processing” status and require polling:

1. Send initial request → get job ID.
2. Poll status API until job is completed.
3. Validate final output.

Example in Postman:

```
let status = pm.response.json().status;
pm.test("Processing started", function () {
  pm.expect(status).to.eql("in_progress");
});
```

D. Scripting & Automation in Postman

41. What scripting language does Postman use for automation?

Postman uses **JavaScript** for both **Pre-request Scripts** and **Tests** tabs.

- **Pre-request Scripts** → Execute before sending the API request (used for dynamic data creation, signing requests, generating tokens).
- **Tests** → Execute after receiving a response (used for validation, storing variables, chaining).

Example Pre-request Script:

```
javascript
CopyEdit
pm.environment.set("timestamp", Date.now());
```

42. How do you store and use dynamic values across requests?

You can extract values from a response and store them in environment or global variables using:

```
javascript
CopyEdit
let token = pm.response.json().token;
pm.environment.set("authToken", token);
```

Then in another request, use:

```
css
CopyEdit
Authorization: Bearer {{authToken}}
```

This allows chaining requests like login → perform action → logout.

43. How do you run data-driven tests in Postman?

Data-driven testing in Postman is done using **Collection Runner** with a CSV or JSON data file.

Steps:

1. Create test data file (users.csv).
2. Use variables in requests ({{username}}, {{password}}).
3. Run collection with data file in Collection Runner.

Example CSV:

```
username,password
user1,pass123
user2,pass456
```

44. How do you write assertions for status codes and response times?

```
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Response time is less than 500ms", function () {
  pm.expect(pm.response.responseTime).to.be.below(500);
});
```

These ensure APIs return correct codes and are within performance SLAs.

45. How do you validate JSON response body in Postman?

```
let jsonData = pm.response.json();
pm.test("Name is John", function () {
  pm.expect(jsonData.name).to.eql("John");
});
```

You can validate:

- Specific fields (`jsonData.id`)
 - Nested objects (`jsonData.address.city`)
 - Arrays (`jsonData.items.length`)
-

46. How do you validate a JSON schema in Postman tests?

```
const schema = {
  "type": "object",
  "properties": {
    "id": { "type": "number" },
    "name": { "type": "string" }
  },
  "required": ["id", "name"]
};

pm.test("Schema is valid", function () {
  pm.response.to.have.jsonSchema(schema);
});
```

This ensures the API response matches the expected contract.

47. How do you log output to Postman Console?

You can debug scripts using:

```
console.log("Token:", pm.environment.get("authToken"));
```

Postman Console is opened with `View → Show Postman Console`.

48. How do you handle loops in Postman scripts?

Postman does not have native looping between requests in one run, but you can:

- Use **Collection Runner** with multiple iterations.
- Use scripts to call `postman.setNextRequest()` for conditional flows.

Example:

```
if(pm.environment.get("retryCount") < 3) {  
  postman.setNextRequest("Check Status");  
}
```

49. How do you call one request from another in Postman?

You can:

- Use `postman.setNextRequest("Request Name")` for chaining.
- Store variables in Pre-request scripts and reuse them.

Example: Login request sets `{{token}}` → used in subsequent APIs.

50. How do you generate random test data in Postman?

```
pm.environment.set("randomEmail", `user${Math.floor(Math.random()*1000)}@test.com`);  
pm.environment.set("randomId", Math.floor(Math.random()*10000));
```

This is useful for avoiding duplicate data issues during test runs.

51. How do you handle conditional testing in Postman scripts?

```
let status = pm.response.code;  
if (status === 200) {  
  pm.test("Valid Response", () => {  
    pm.expect(pm.response.json().status).toEqual("success");  
  });  
} else {  
  pm.test("Error Response", () => {  
    pm.expect(status).toBeOneOf([400, 404]);  
  });  
}
```

This ensures flexible validation depending on API behavior.

52. How do you automate Postman tests in CI/CD pipelines?

1. Export the Postman collection and environment.
2. Install **Newman** in your CI/CD environment.
3. Add a build step:

```
newman run MyCollection.json -e MyEnv.json --reporters cli,html
```

4. Fail the build if Newman tests fail.
-

53. How do you capture values from headers in Postman?

```
let contentType = pm.response.headers.get("Content-Type");  
pm.environment.set("contentType", contentType);
```

This is useful for APIs that send tokens in headers instead of body.

54. How do you validate arrays in Postman response?

```
let jsonData = pm.response.json();  
pm.test("Array contains 'Apple'", function () {  
    pm.expect(jsonData.items).to.include("Apple");  
});  
pm.test("Array length is 5", function () {  
    pm.expect(jsonData.items.length).to.eql(5);  
});
```

55. How do you run only selected tests in a Postman collection?

- Use **folders** and run them in Collection Runner.
 - Temporarily disable other requests.
 - Use `postman.setNextRequest(null)` to stop execution after certain tests.
-

56. How do you use environment variables in scripts?

```
pm.environment.set("userId", pm.response.json().id);  
let id = pm.environment.get("userId");
```

Environment variables make requests portable between Dev, QA, and Prod.

57. How do you parameterize requests in Postman?

```
{{baseUrl}}/users/{{userId}}
```


These placeholders are replaced by variable values at runtime.

58. How do you run Postman tests headlessly?

Use **Newman CLI**:

```
newman run mycollection.json -e myenv.json --reporters cli,json
```

This is essential for automation and CI/CD.

59. How do you retry failed requests in Postman scripts?

Postman itself doesn't have automatic retries, but you can simulate:

```
if (pm.response.code !== 200) {  
  postman.setNextRequest("Same Request Name");  
}
```

Limit retries with a counter variable.

60. How do you test APIs with file uploads in Postman?

- Use **form-data** in the Body tab.
 - Select file type and attach the file.
 - Validate upload success in response.
-

E. Advanced Testing Scenarios

61. How do you test versioned APIs?

API versioning ensures backward compatibility when new changes are introduced.

Steps:

1. Identify available versions (`/v1/`, `/v2/`).
2. Compare responses for the same request in each version.
3. Ensure old clients still function with older versions.
4. Validate deprecation notices in headers or documentation.
5. Ensure new fields are optional and don't break existing responses.

Example:

```
GET /v1/users/123 returns {name, email}
```

```
GET /v2/users/123 returns {name, email, phone} — old apps should still work without phone.
```

62. How do you test APIs in a microservices architecture?

Challenges:

- APIs may depend on multiple services.
- Failures in one service can cascade.

Testing strategy:

- Test each service independently with mock data.
- Use **contract testing** (e.g., Pact) to verify integrations.
- Test end-to-end workflows to ensure orchestration is correct.

Example: In an e-commerce app, the **Order Service** API depends on **Inventory Service** and **Payment Service**.

63. How do you test APIs that return asynchronous results?

Some APIs process requests in the background and return job IDs.

Steps:

1. Send initial request (POST).
2. Receive `jobId` and `status pending`.
3. Poll status API until completion.
4. Validate final output or error.

Example in Postman:

```
let status = pm.response.json().status;
if (status !== "completed") {
  postman.setNextRequest("Check Job Status");
}
```

64. How do you test APIs with third-party integrations?

Third-party APIs can be slow, expensive, or unavailable in testing.

Solutions:

- Use **mock servers** to simulate their responses.
 - Validate API error handling when the third party fails.
 - Verify that the correct payload is sent to the external API.
-

65. How do you test API rate limiting?

Rate limits prevent abuse.

Steps:

1. Identify rate limit policy (e.g., 100 requests/min).
2. Send rapid requests until limit is hit.

3. Validate correct error response (often 429 Too Many Requests).
 4. Check `Retry-After` header for cooldown time.
-

66. How do you test webhooks in Postman?

Webhooks send data to your endpoint when an event happens.

Testing:

- Use Postman's **Mock Server** as the webhook receiver.
 - Send a test event from the service.
 - Validate payload and headers received.
-

67. How do you test GraphQL APIs in Postman?

Steps:

1. Set request type to POST.
2. In Body → select GraphQL.
3. Write query/mutation:

```
query {  
  user(id: "123") {  
    name  
    email  
  }  
}
```

4. Validate that only requested fields are returned.
-

68. How do you test APIs with file downloads?

- Send request.
 - Check `Content-Disposition` header for filename.
 - Verify file type (`Content-Type`).
 - Save and compare file size/hash to expected.
-

69. How do you test APIs that require digital signatures?

- Use Pre-request Scripts to generate HMAC or RSA signatures.
- Use `CryptoJS` in Postman to hash request payload.

Example:

```
const hash = CryptoJS.HmacSHA256(pm.request.body.raw, "secretKey");  
pm.environment.set("signature", hash.toString(CryptoJS.enc.Hex));
```

70. How do you mock APIs in Postman?

Postman Mock Server steps:

1. Create a new mock server.
 2. Define sample responses for endpoints.
 3. Use mock URL in place of real API.
This is useful for testing UI before backend is ready.
-

71. How do you handle API request sequencing in Postman?

- Use `postman.setNextRequest()` to define next request order.
 - Store values from previous responses in variables.
 - Example: Login → Create User → Get User → Delete User.
-

72. How do you test APIs that have different behavior for different users?

- Use different authentication tokens for different roles (admin, user, guest).
 - Validate role-based access control (RBAC).
 - Example: Admin can `DELETE /users/123`, normal user cannot.
-

73. How do you test an API migration from monolith to microservices?

- Identify API endpoints before and after migration.
 - Compare responses for the same input.
 - Test for **performance improvements** and **no functional breakage**.
 - Validate integration points with new microservices.
-

74. How do you test an API that returns streaming data?

Streaming APIs (e.g., WebSocket, SSE):

- Connect to the stream in Postman (WebSocket tab).
 - Send a message and validate incoming stream messages.
 - Check message format and frequency.
-

75. How do you test APIs with scheduled jobs?

If API triggers a scheduled task (e.g., nightly report):

- Use a smaller schedule in test environment.
- Trigger job manually if API supports it.
- Validate job status and output.

76. How do you handle flaky APIs in automation?

- Add retry logic in scripts.
- Validate response before proceeding to next test.
- Log failures for further analysis.

Example:

```
if (pm.response.code !== 200) {  
    postman.setNextRequest("Same Request");  
}
```

77. How do you test localization in APIs?

- Send requests with `Accept-Language` header (en-US, fr-FR).
 - Validate responses are translated correctly.
 - Check currency and date formats per locale.
-

78. How do you validate API caching behavior?

- Check `Cache-Control` and `ETag` headers.
 - Send repeated requests and confirm whether server returns cached or fresh data.
-

79. How do you test APIs under failure scenarios?

- Simulate server errors (mock 500 response).
 - Simulate network delays/timeouts.
 - Validate application behavior on API failure.
-

80. How do you test gRPC APIs in Postman?

Postman now supports gRPC:

- Import `.proto` file.
- Select method to test.
- Send request with parameters.
- Validate response fields.

F. CI/CD & Integration Testing

81. How do you integrate Postman tests into CI/CD pipelines?

- Export Postman collection & environment as JSON.
- Install **Newman** (Postman CLI).
- In CI/CD (Jenkins, GitHub Actions, Azure DevOps), add a step to run:

```
newman run collection.json -e environment.json
```

- Fail build if any test fails (`--bail`).
-

82. What is Newman and why is it used?

- CLI tool to run Postman collections without opening the UI.
 - Used in automation, CI/CD, and scheduled jobs.
 - Supports HTML, JSON, and JUnit reports.
-

83. How do you generate HTML reports from Newman?

Install reporter:

```
npm install -g newman-reporter-html
```

Run with:

```
bash
CopyEdit
newman run collection.json -r html
```

84. How do you schedule Postman collections to run automatically?

Options:

- Use **Newman + cron jobs** on a server.
 - Use **Postman Monitors** (cloud execution on a schedule).
 - Integrate with **Jenkins** to run daily/nightly.
-

85. How do you pass dynamic environment variables in CI/CD?

- Use environment JSON with placeholders.
- Override variables with `--env-var` flag in Newman:

```
newman run collection.json --env-var baseUrl=https://staging.api.com
```